

EXAMEN FINAL TEÓRICO - 2025/2

Curso: Estructuras de datos Avanzadas

Pregunta 1 (Outcomes 1,2,6)(10 puntos):

En el mundo mágico, el Ministerio de Magia te ha encomendado una tarea inusual: crear el mayor escudo protector posible usando varitas antiguas encontradas en el Bosque Prohibido. Cada varita tiene una longitud entera positiva y todas ellas están guardadas en un cofre, representado por el vector `nums` de tamaño `n`.

Tu misión es construir el mayor escudo protector posible, formando un polígono con algunas de estas varitas.

Pero todo escudo mágico debe respetar dos reglas sagradas:

1. Debe tener al menos 3 lados.
2. La varita más larga no puede superar la suma de las demás; de lo contrario, el hechizo se rompe.

Debes completar la función: `long long largestPerimeter(vector<int>& nums) { . . . }` para determinar cuál es el mayor perímetro que puede alcanzarse utilizando las varitas en `nums`. Si ninguna combinación permite crear un escudo estable, devuelve `-1`.

Ejemplo 1

Entrada: `nums = [5, 5, 5]`

Salida: 15

Explicación: El único polígono posible que se puede formar con `nums` tiene 3 lados: 5, 5 y 5. El perímetro es $5+5+5=15$.

Ejemplo 2

Entrada: `nums = [1, 12, 1, 2, 5, 50, 3]`

Salida: 12

Explicación: El escudo más grande se forma con las varitas `[1, 1, 2, 3, 5]`. El perímetro es $1+1+2+3+5=12$. No podemos usar las varitas de 12 y 50 como lado mayor, pues no existen varitas menores cuya suma las supere.

Ejemplo 3

Entrada: `nums = [5, 5, 50]`

Salida: -1

Explicación: No es posible formar un polígono con `nums`, ya que el escudo debe tener al menos 3 lados, y $50 > 5+5$.

Indica la complejidad de tu solución. **El ejercicio tendrá una nota de 0 en caso de que el código entregado no compile.**

Pregunta 2 (Outcomes 1,2,6)(10 puntos):

Escriba el código para implementar los algoritmos de clustering Kmeans y DBScan, y proponga una función que nos permita encontrar buenos/optimos valores de nuestros parámetros de entrada para mejorar el rendimiento del algoritmo DBScan.

(3 puntos) Funcion(es) para ejecutar Kmeans CORRECTAMENTE desde la función main()

(4 puntos) Funcion(es) para ejecutar DBScan CORRECTAMENTE desde la función main()

(3 puntos) Función de pre-procesamiento para DBScan, funcionando y CORRECTAMENTE justificada/explicada con comentarios.

Observaciones:

- El código debe ser probado con al menos 50 puntos aleatorios. Puede usar la siguiente función para generar los puntos aleatorios

```
#include <cmath>

using namespace std;

float randomFloat()
{
    return (float)(rand()) / (float)(rand());
};
```